

Итерация и циклы.
Работа со строками.
Списки и кортежи.

Основы языка программирования Python

Лекция 2

Преподаватель: Моисеева Анна Юрьевна

Структурирование: понятия

1.
2.



Последовательность

Команды выполняются в том порядке, в котором они представлены в коде

if



Ветвление

Условные конструкции, создающие ветки



Цикл

Команды повторяются либо заданное кол-во раз, либо до тех пор, пока условие равно True

Цикл for

Синтаксис цикла:

```
for <переменная> in <итерируемый объект>:  
    команда  
    команда  
else:  
    команда
```

Циклу **for** нужен **итерируемый объект** — объект, который может возвращать элементы **по одному**.

Такими объектами являются, например, **строки**

Используется, когда:

- Мы **знаем, сколько раз** нам повториться,
- Мы перебираем элементы **итерируемого объекта**.

Пример такого цикла:

```
for letter in "abcd":  
    print(letter)
```

a
b
c
d

Диапазоны и range()

```
for i in range(3, 0, -1):  
    print(i)
```

3
2
1

Range

Конец не входит, диапазон идет до предыдущего числа – если шаг отрицательный, до того, которое больше.

Команда `range()` создает диапазон чисел – еще один тип итерируемого объекта.

Синтаксис:

`range(<начало>, <конец>, <шаг>)`

Можем задать и меньше параметров:

`range(<конец>)` от 0 или `range(<начало>, <конец>)`

ВНИМАНИЕ!

Счет по порядку программисты всегда начинают с нуля.

Цикл while

Синтаксис цикла:

```
while <условие>:
```

```
    команда
```

```
    команда
```

```
else:
```

```
    команда
```

Циклу **while** нужно **условие**, которое будет заставлять его повторять команды. Если условие будет **False**, сработает **else**.

Синтаксис **while** отчасти похож на синтаксис **if**

Используется, когда:

- Мы **не** знаем, сколько раз нам повториться,
- Есть **условие выполнения** для продолжения цикла.

Пример такого цикла:

```
number = input()
while not number.isdigit():
    number = input('Введите число!\n')
else:
    print('Наконец-то число!')
```

Вложенные циклы

```
string = input("Введите строку: ")
while string:
    for letter in string:
        print(letter)
    string = input("Введите строку: ")
else:
    print("Цикл закончен")
```

```
Введите строку: эй
э
й
Введите строку:
Цикл закончен
```

Цикл **while** запрашивает строку до тех пор, пока пользователь не нажимает на Enter (это будет пустая строка, и **bool("") = False**).

Цикл **for** печатает эту строку побуквенно.

Когда пользователь ввел пустую строку, программа печатает «**Цикл закончен**».

Отступы в Python

```
string = input("Введите строку: ")
while string:
    if len(string) <= 2:
        for letter in string:
            print(letter)
        else:
            print(f"Мы напечатали букв: {len(string)}")
    else:
        print("Слишком много букв")
    string = input("Введите строку: ")
else:
    print("Цикл закончен")
```

Красное – цикл
while

Синее – проверка
Зеленое – цикл
for

while True

Исполнение цикла `while` требует bool, возвращающего `True`. Таким образом, если мы зададим неизменяемое условие наподобие `while True`, то получим **бесконечный цикл**, из которого сможем выйти только с помощью команды `break` или вручную прекратив исполнение кода.

`while False`, как можно догадаться, никогда не выполнится.

ОПАСНО!

```
spisok = []  
while True:  
    spisok.append("a")
```

Подобный код будет бесполезно и беспощадно грузить Python-ядро, а также бесконечно потреблять оперативную память (с каждой итерацией все больше). Может вызвать **зависание компьютера** или **вылет из системы**.

break и continue

```
while True:
    number = input("Введите число: ")
    if number.isdigit():
        print("Молодчина!")
        break
    print('Я сказал число!') break
```

Команда **break** выводит из цикла, прерывая его работу.

Команды, следующие за ней в теле цикла, **не выполняются**.

Следующая итерация цикла **не наступает**.

Команда **continue** прерывает только текущую итерацию и переходит к следующей. То, что следует за **continue**, **не выполнится**, но цикл не остановится.

Обе команды **применимы** как для цикла **while**, так и для цикла **for**.

```
while True:
    number = input()
    if number.isdigit():
        break
    if number.isspace():
        print('нам не нужен пробел!')
        continue
    print('Вы ввели букву!') continue
```

Строка в Python

str - последовательность **СИМВОЛОВ**.
str неизменяема, но **итерируема** (ее
элементы можно перебрать по частям).



Итерируем:

```
[18] string = "КОТ"  
     for sym in string:  
         print(sym)
```

K
O
T

Попробуем изменить символ в строке:

```
string = "КОТ"  
string[0] = "В"
```

```
-----  
TypeError                                 Traceback (most recent call last)  
<ipython-input-12-0583f8633c5a> in <cell line: 2>()  
      1 string = "КОТ"  
----> 2 string[0] = "В"
```

```
TypeError: 'str' object does not support item assignment
```

Вместо этого:

```
string = "КОТ"  
print("В" + string[1:])
```

ВОТ

Закавычивание строк

'Hello'
"""Hello""" Vs """Hello"""

Одинаковое количество кавычек с обеих сторон (**двойных** или **одиночных**) говорит Python, что перед нами строка

Переносы:



```
print("""Смешной
кот""")
```

Смешной
кот

```
print("Смешной\nкот")
```

Смешной
кот

Апострофы:

```
'Kitty's fur'
```

File "<ipython-input-19-8de6dea8a63e>"

```
'Kitty's fur'
```

^

SyntaxError: unterminated string literal



"Kitty's fur"



'Kitty's fur'

Комментарии:

```
'''Сколько будет 2 + 2? Многострочный
Это комментарий.'''
```

```
# И предыдущая строка, и эта - комментарии.
2 + 2 Однострочный
```

4

Символ \ - применение

Экранирование		Спец. последовательности	
\\	Один обратный слэш.	\n	Новая строка.
\'	Один апостроф.	\t	Горизонтальная табуляция.
\"	Одна кавычка.	\r	Возврат каретки.
Обозначения			
\u	16-битовый символ Юникода.	\U	32-битовый символ Юникода.
\x	16-ричное значение	\o	8-ричное значение

В случае **экранирования** бэкслэш говорит нам, что перед нами — **последовательность символов**, а не элемент **синтаксиса** Python.

Срезы

`s = 'abcdefghij'`

`s[x:y:z]`

x - начало, y - конец, z - шаг

0	1	2	3	4	5	6	7	8	9
a	b	c	d	e	f	g	h	i	j

Код		Результат	
<code>s[0]</code>	<code>'a'</code>	a bcdefghi	Возвращает нулевой символ в строке.
<code>s[1:4]</code>	<code>'bcd'</code>	a bcd efghij	Возвращает все символы с 1 до 3.
<code>s[1:4:2]</code>	<code>'bd'</code>	a bcd efghij	Возвращает все символы с 1 до 3 через один.
<code>s[:3]</code>	<code>'abc'</code>	abc defghij	Возвращает первые три символа.
<code>s[2::2]</code>	<code>'cegi'</code>	ab cd efghi	Возвращает все символы со 2 до конца через один.
<code>s[-1]</code>	<code>'j'</code>	abcdefghi j	Возвращает первый символ с конца.
<code>s[::-5:-1]</code>	<code>'jihg'</code>	abcdef ghij	Возвращает последние 4 символа в обратном порядке

Заметим, что правый край среза не входит в последовательность (как в range)

Префиксы строк

Преф.	Результат		
r (raw)	<code>r'd:\games\u0026x28;nexus'</code>	<code>d:\games\nexus</code>	Отменяет экранированные последовательности. Идеально для путей и регулярных выражений.
b (bytes)	<code>b'abc'</code> <code>b[0]</code> <code>b[1]</code>	<code>b'abc'</code> 97 98	Делает строку последовательностью байтов. Возвращает значение только при обращении по индексу.
f (format -ted)	<code>f'{1+1} это 2'</code>	2 это 2	Строка с возможностью вставить в нее значение переменной или выражения.

Префиксы можно спокойно сочетать. Например:

`rf'{path}\train'`

Операции сравнения для строк

Операции сравнения применяются к строкам так:

- мы сравниваем строки **посимвольно**: доходим до **первого символа**, который в них различен, и применяем операцию сравнения **к этому символу**;
- мы сравниваем не сами символы, а числа в 16-ричной СС – **их коды** в **Unicode** (кодировке).

Код "a" – это 97,
код "b" – это 98,
код "d" – это 100.

97 == 97,
но 98 < 100.
T.e. string1 < string2

```
string1 = "ab"  
string2 = "ad"  
print(string1 < string2)
```

True

Функции ord() и chr()

`ord(char)`

Возвращает код символа в кодировке (UTF-8)

`ord()` делает ровно то же самое, что и префикс `b` с индексом для байтовых строк.

```
print(ord("a"))
```

97

```
print(chr(97))
```

a

`chr(code)`

Возвращает символ по его коду

`ord()` и `chr()` – это взаимно обратные функции.

Функции и методы Python

`print()` — функция. Она может применяться к объектам разных типов — к примеру, как к `int`, так и к `str`.

Функция `print()` в документации очень укороченно выглядит так:
`print(*args, sep="", end="")`
`*args` — неопределенное кол-во аргументов, данное в определенной последовательности.

`.strip()` — метод. Он применяется только к объектам типа `str`. Это по сути функция, привязанная только к классу `str`.

```
print("abc", 1)
print(" abc ".strip())
number = 1
number.strip()
```

```
abc 1
abc
```

`AttributeError`

Длина строки – функция len()

```
s = 'abcdefg'  
len(s)
```

7

```
print(s[len(s) - 1])
```

g

`len(string)`

Возвращает количество символов в строке.

Так как `len()` - это функция, она может применяться как к строкам, так и к другим объектам Python: спискам, множествам и словарям.

Заметим, что индекс последней буквы – не 7, а 6, потому что считаем с 0.



Методы строк, возвращающие строки

Метод	Результат	
<code>'abc'.replace('a', 'b')</code> <code>'abc'.replace('ab', 'cb')</code>	bbc cbc	Заменяет символы. Можно заменить сразу несколько символов.
<code>' abc '.strip()</code> <code>'1abc1'.strip('1')</code>	abc abc	Отрезает приведенные в аргументах символы слева и справа. По умолчанию отрезает все пробельные символы.
<code>'1abc1'.rstrip('1')</code>	1abc	Отрезает символы только справа.
<code>'1abc1'.lstrip('1')</code>	abc1	Отрезает символы только слева.
<code>'aBc'.upper()</code>	ABC	ВЕРХНИЙ РЕГИСТР.
<code>'ABc'.lower()</code>	abc	нижний регистр.
<code>'abc'.capitalize()</code>	Abc	Первая заглавная буква.
<code>'ab c'.title()</code>	Ab C	Все Первые Буквы Заглавные.

Методы строк, возвращающие числа

Метод	Результат	
<code>'abc'.find('b')</code> <code>'abc'.find('ab')</code>	1 0	Поиск символов с начала. Возвращает индекс первого совпадения. Если не найдено, возвращает -1.
<code>'abab'.rfind('a')</code> <code>'abab'.rfind('ab')</code>	2 2	Поиск символов с конца. Возвращает индекс первого совпадения. Если не найдено, возвращает -1.
<code>'abc'.index('a')</code> <code>'abc'.index('d')</code>	1abc ValueError	Работает так же, как find. Используется только в случае, если точно известно, что символ присутствует.
<code>'abab'.count('a')</code> <code>'abab'.count('ab')</code>	2 2	Считает, сколько раз встречаются символ или последовательность символов в строке.

Помним, что в программировании индексы начинаются с 0.

Проверки: возвращают bool

Метод	Результат	
<code>'abc'.startswith('ab')</code>	True	Начинается ли строка с этих символов?
<code>'abc'.endswith('ab')</code>	False	Заканчивается ли строка этими символами?
<code>'ABc'.isupper()</code>	False	Все буквы заглавные?
<code>'abc'.islower()</code>	True	Все буквы строчные?
<code>'Abc B'.istitle()</code>	True	Все первые буквы заглавные?
<code>'1234'.isdigit()</code>	True	Строка состоит только из цифр?
<code>'ab1'.isalpha()</code>	False	Строка состоит только из букв?
<code>' '.isspace()</code>	True	Строка содержит только пробельные символы?
<code>'ab123'.isalnum()</code>	True	Строка состоит из цифр и/или букв?

Операция in

ch in string

Возвращает bool: есть ли в строке этот символ?

```
if "a" in "abcd":  
    print('"a" есть в строке.')
```

"a" есть в строке.



True

```
print("a" not in "abcd")
```

False

False



Списки в Python

```
ints = [1, 2, 3]           # Список int
strs = ['1', '2', '3']     # Список str
lists = [[1], [2], [3]]    # Список списков
mixed = [1, "2", [3]]       # Смешанный список
```

Список (list) –
итерируемый
изменяемый
объект.

- Все элементы в списке **индексируются** (как в строке).
- К спискам применимы **срезы**.
- Список может включать в себя объекты **любых типов**.
- Список может состоять из объектов **разных типов**.
- **Порядок** элементов в списке **строго определен**.
- В списке могут быть **повторяющиеся** элементы.



Операции со списками

```
lst = [1, 2, 3] + [4, 5, 6]  
lst
```

```
[1, 2, 3, 4, 5, 6]
```

```
lst = ["a", "h"] * 3  
lst
```

```
['a', 'h', 'a', 'h', 'a', 'h']
```

Списки можно **складывать**
и **умножать** на число.

Результат такой же, как и при
сложении и умножении
строк.



Операции со списками

Для обращения к отдельному элементу списка используем индексы

```
lst = [2, 3, "cat", ["a"]]
lst[0] *= 2
lst[1] = lst[0] - lst[1]
lst[2] = lst[2].replace("c", "r")
lst[3].append("b")
lst

[4, 1, 'rat', ['a', 'b']]
```

К элементу списка можно применять любые **операции, функции и методы**, которые применимы к **типу данных** этого элемента.

map(func, lst) – функция, принимающая в качестве аргумента **другую функцию** и применяющая ее подряд к **каждому элементу списка**.

Добавление элемента в список

```
lst = [1, 2, 3]
lst.append(4)
lst.append("5")
lst.append([6, 7])
lst
```

```
[1, 2, 3, 4, '5', [6, 7]]
```

`list.append()` позволяет добавить **один** элемент в конец списка.

Если мы передаем туда **список**, то он **не расширяет** текущий, а добавляется в качестве **элемента**.

Получается список внутри списка.

Удаление элемента из списка

```
lst = [1, 2, 3, 4, 5]
del lst[0] # с помощью del
lst[0:1] = [] # с помощью срезов
elem = lst.pop(0) # с помощью pop
print(elem) # pop вернул удаленный элемент
lst.remove(4) # с помощью remove (по значению)
lst
```

```
3
[5]
```

Для удаления элемента по индексу: `del`, срезы и `pop`.

Метод `.pop()` не просто удаляет элемент, но и **возвращает** его.

Для удаления по значению есть метод `.remove()`. Он удаляет **первое** вхождение элемента. Если не находит — **ValueError**.

* `del` - это не просто оператор списков, это «магический» метод работы с объектами Python

Вставка элемента в список

```
lst = ["a", "c"]  
lst.insert(1, "b")  
lst
```

```
['a', 'b', 'c']
```

```
lst = ["a", "c"]  
lst[1:1] = ["b"]  
lst
```

```
['a', 'b', 'c']
```

`list.insert(index, elem)` – метод списков для вставки элемента по индексу.

`list[index1:index2] = [elem]` – вставка с помощью срезов. Можно добавить сразу несколько элементов.

Подсчет вхождений элемента в список

```
lst = [1, 1, 2, 3, 1]
print(lst.count(1))
lst = ["a", "a", "b"]
print(lst.count("a"))
```

3

2

`list.count(elem)`

Метод `count` позволяет подсчитать количество вхождений элемента в список.

`.count()` работает со списками в целом так же, как со строками, но только со строками есть еще возможность искать в строке подстроку; со списками такой возможности нет.

Поиск элемента в списке

```
lst = [1, 2, 3, 4]
print(lst.index(3))
lst.index(8)
```

2

ValueError

`list.index(elem)`

Метод `.index()` позволяет получить индекс элемента в списке. Если не найдено, то возвращается **ValueError**.

Чтобы использовать `.index()` в тех случаях, когда мы не уверены, что элемент есть в списке, можно предварительно проверять его наличие с помощью оператора `in`.

Расширение списка

```
lst = [1, 2, 3]
```

```
lst += [4, 5]
```

```
lst
```

```
[1, 2, 3, 4, 5]
```

```
lst = [1, 2, 3]
```

```
lst.extend([4, 5])
```

```
lst
```

```
[1, 2, 3, 4, 5]
```

`list1.extend(list2)`

Метод позволяет **вставить** элементы из второго списка в конец первого. Работает так же, как и оператор **+**, только делает это **in-place** (т.е. тут же, без необходимости класть результат в переменную).

In-place методы, если их результат положить в переменную, возвращают **None**.

Методы строк .split() и .join()

```
lst = "cat dog ball".split()  
lst
```

```
['cat', 'dog', 'ball']
```

```
lst = "cat, dog, ball".split(", ")  
lst
```

```
['cat', 'dog', 'ball']
```

Метод строк `.split()` позволяет **разрезать** строку по указанному разделителю и создать из нее **список строк**.

По умолчанию разделитель – это пробельный символ.



Методы строк `.split()` и `.join()`

```
lst = ["a", "b", "c"]  
" ".join(lst)
```

'a b c'

```
lst = ["Коля", "Витя", "Саша"]  
" и ".join(lst)
```

'Коля и Витя и Саша'

Метод строк `.join()` позволяет **соединить список строк** в одну строку, используя указанный **разделитель**.

Методы `.split()` и `.join()` имеют очень много применений при работе с **текстами**; особенно они полезны в связке с функцией `map()`.

Функции, применимые к спискам

Функция	Результат	Описание
<code>len([1, 2, 3])</code>	3	Возвращает длину списка (сколько в нем элементов)
<code>min([2, 1, 3])</code>	1	Возвращает самый маленький элемент
<code>max([1, 3, 2])</code>	3	Возвращает самый большой элемент
<code>sum([1, 2, 3])</code>	6	Возвращает сумму всех элементов (если все элементы - числа)

Сравнение списков

$[1, 2, 3] < [3, 4, 5]$

$[1, 2, 3] < [1, 3, 4]$

$[1, 1, 3] < [1, 1, 4]$

$[1, 1, 1] == [1, 1, 1]$

Операции сравнения применяются к спискам так же, как к строкам :

- мы сравниваем списки **поэлементно**;
- неравенство появляется, как только находим **первый различный** элемент;
- сами элементы мы сравниваем в соответствии с их **типом данных**.

Когда сравниваем списки списков, имеем **рекурсию**

Копирование списка

```
x = 5
y = x
x += 1
print(x, y)
```

6 5

Числа
(неизменяемый
объект)

```
lst1 = [1, 2]
lst2 = lst1
lst2.append(3)
print(lst1, lst2)
```

[1, 2, 3] [1, 2, 3]

Списки
(изменяемый
объект)

ВНИМАНИЕ!

Если объект **изменяемый**,
то Python при присвоении
этого объекта другой
переменной создаст
ссылку на первоначальный
объект.

Изменив **мнимую копию**,
изменим и
оригинальный объект.

Копирование списка

```
lst1 = [1, 2]
lst2 = lst1[:]
lst2.append(3)
print(lst1, lst2)
```

```
[1, 2] [1, 2, 3]
```

```
lst1 = [1, 2]
lst2 = lst1.copy()
lst2.append(3)
print(lst1, lst2)
```

```
[1, 2] [1, 2, 3]
```

Копию списка можно сделать

- либо с помощью **пустого среза**:

```
list1 = list2[:]
```

- либо с помощью метода

.copy():

```
list1 = list2.copy()
```



Сортировка списка

```
lst = [1, 4, 3, 2]  
lst.sort()  
lst
```

```
[1, 2, 3, 4]
```

```
lst = [1, 4, 3, 2]  
lst = sorted(lst)  
lst
```

```
[1, 2, 3, 4]
```

Сортировать список можно:

- либо с помощью **in-place** метода **.sort()**;
- либо помощью функции **sorted()** — это **не in-place** функция, так что ее результат надо куда-то класть.

sorted() применима к **любым итерируемым объектам**, но возвращает всегда **список**.

Обращение списка

```
lst = [1, 2, 3]
lst.reverse()
lst
```

```
[3, 2, 1]
```

```
lst = [1, 2, 3]
lst = reversed(lst)
print(lst)
print(list(lst))
```

```
<list_reverseiterator object at 0x7f2301b5b4c0>
[3, 2, 1]
```

Перевернуть список можно:

- либо с помощью in-place метода `.reverse()`.
- либо с помощью функции `reversed()`, которая возвращает `reverseiterator` — его можно легко превратить в список с помощью `list()`.

Также не забываем про срезы!

Кортежи

```
tpl1 = (1, 2, 3)
tpl2 = 1, 2, 3
print(type(tpl1), type(tpl2))
```

```
<class 'tuple'> <class 'tuple'>
```

```
tpl1[0] = 5
```

TypeError

Кортеж (**tuple**) похож на список, но, в отличие от последнего, **неизменяемый**.

Кортежи являются **hashable** объектом.
Они хороши, когда нам нужен **неизменяемый набор констант**.

Для кортежей есть всего два встроенных метода: **.count()** и **.index()**, работают как у списка.

Спасибо за внимание!